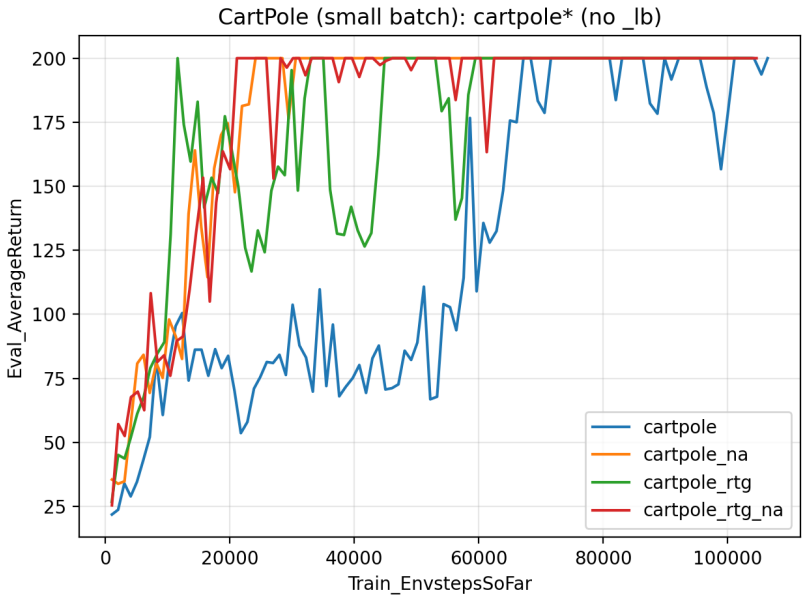
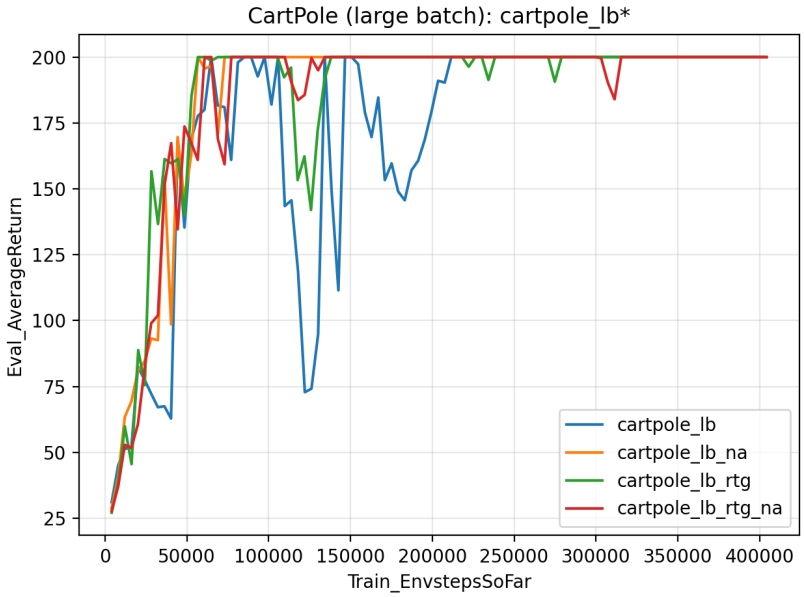
Experiment1

1. Learning Curves





2. Analysis
   a) The one using reward-to-go outperforms the one using trajectory-centric estimator in the cases without advantage normalization, as in both graphs reward-to-go (green) reaches high return earlier and exhibits slighter oscillation than the trajectory-centric estimator (blue), especially in the small-batch setting.
   b) The reward-to-go estimator is usually preferred over trajectory-centric

estimator. This is because both estimators are unbiased, but the reward-to-go estimator removes past rewards that are independent of the current action from the policy gradient weight. This reduces variance of the gradient estimator and typically improves learning speed and stability.

c) The advantage normalization trick helps, as in in both plots, the normalized variants (orange vs. blue; red vs. green) generally reach high returns earlier and exhibit reduced oscillation.

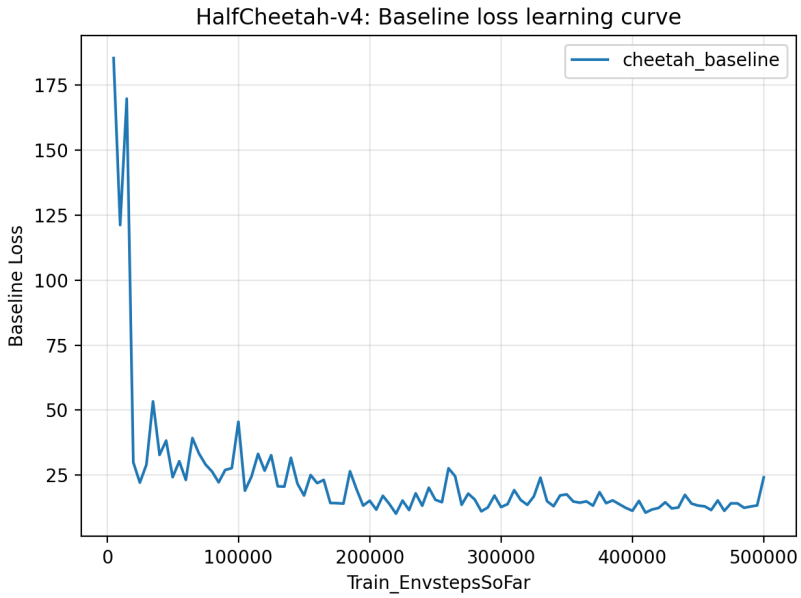
d) The batch size does make an impact, as in two graphs generally the smaller batch size performs faster improvement but is more unstable (more oscillation). This is because with smaller batches, the agent performs more frequent updates per environment step, which can speed up early learning, but typically with higher variance gradients. Larger batches generally produce smoother and more stable learning curves but may improve more slowly early on due to fewer updates.

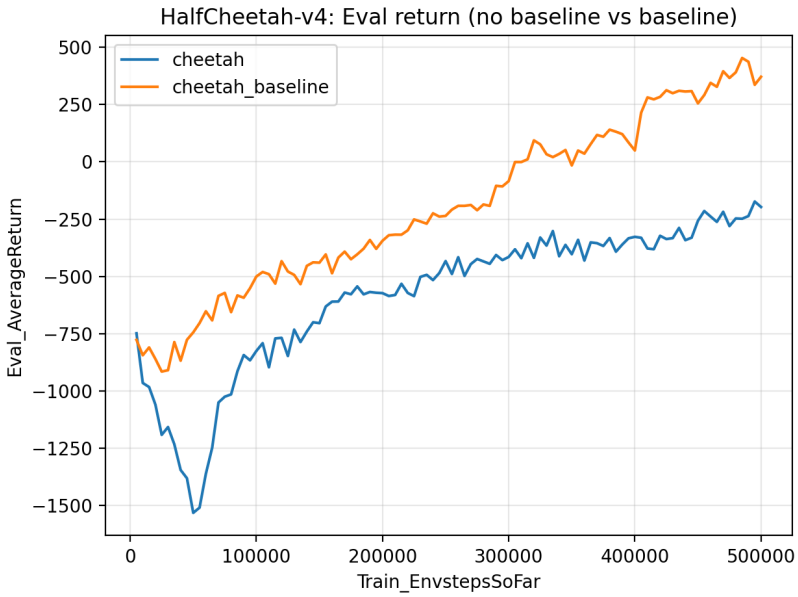
3. Command line configurations:

I used the same command line configurations as provided in homework instructions.

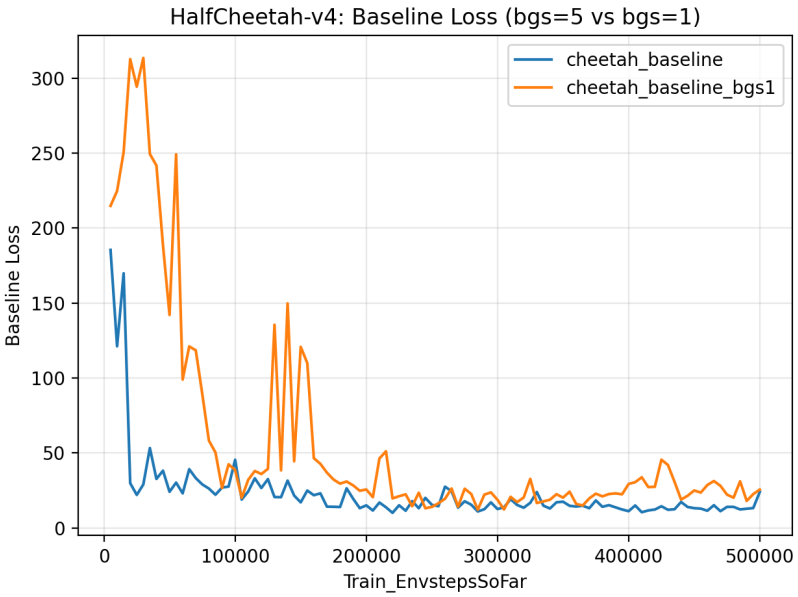
Experiment2

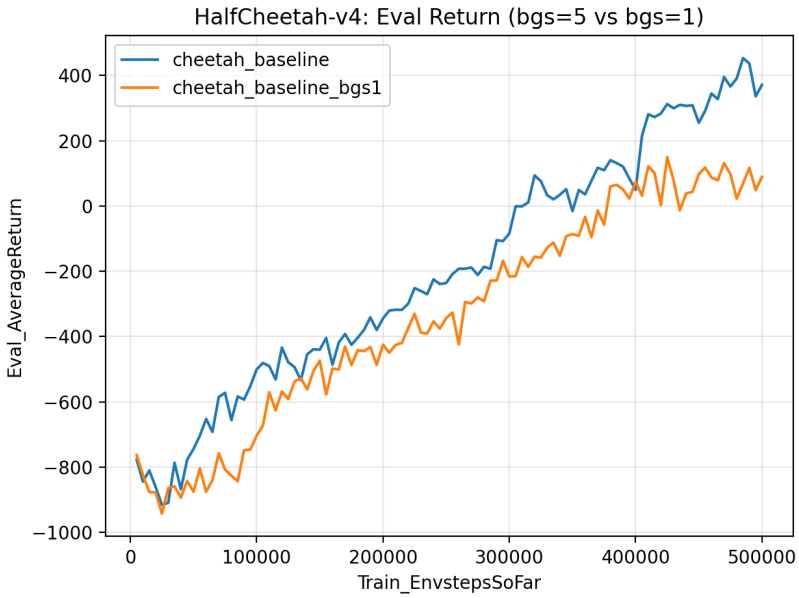
1. Learning Curves





2. Reduced baseline gradient steps



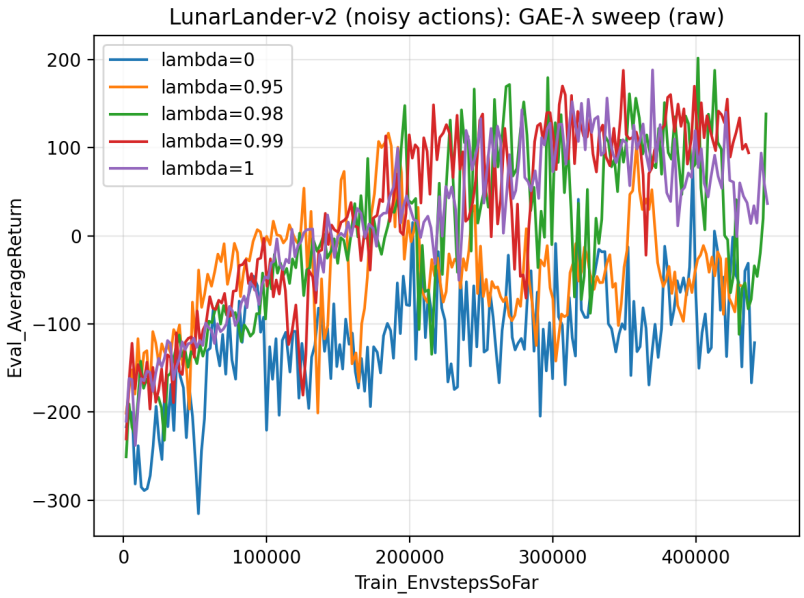


From the plots, reducing the baseline gradient steps from 5 to 1 makes the critic learn less effectively. We can observe that the baseline loss stays higher and shows larger oscillations, indicating poorer value-function fitting. With a weaker baseline, the advantage estimates become noisier, which increases the variance of the policy gradient updates. As a result, the policy learns more slowly and reaches a substantially lower final eval return compared with the bgs=5 setting.

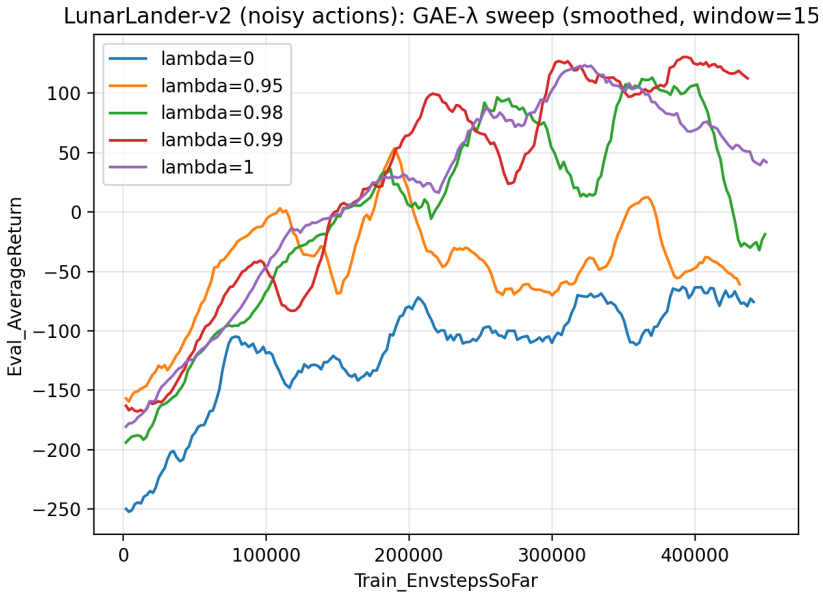
3. Command line configurations:
   I used the same command line configurations as provided in homework instructions, except for that I reduced the baseline gradient steps from 5 to 1 in (2).

Experiment3

1. Plot & analysis
   I plotted the curves of evaluation average return w.r.t different lambda values in a single graph, and it is shown that the best policy has achieved a return above 150 at least once during training:



The above result is too noisy, so I applied a sliding window smoothening for better visualization:



We can see that lambda = 0 leads to the worst performance due to strong critic bias. The result is gradually improved as lambda increases to 0.95 and 0.98, as they reduced the bias. The best result is shown at lambda = 0.99, where the bias and variance achieved a good balance. The performance decreased a bit at lambda = 1 due to larger variance.

2. When lambda = 0, the GAE degenerate to one step TD advantage that is used by classical Actor-Critic algorithm. When lambda = 1, the GAE is a Monte Carlo advantage estimator. The one step TD advantage relies heavily on critic estimation, which introduces large bias when the critic isn't converged at early stage, so lambda = 0 demonstrates the worst result. When lambda = 1, the

Monte Carlo estimation introduces more variance. So, lambda = 1 shows a decreased performance compared with lambda = 0.99 where the variance and bias are best balanced.

3. Command line configurations:

   I used the same command line configurations as provided in homework instructions.

Experiment4

1. Best set of hyperparameters:

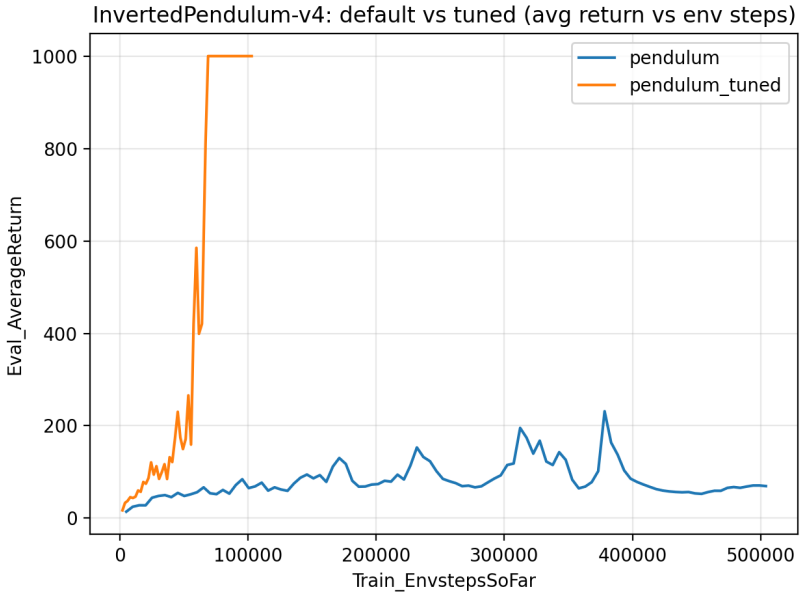
Batch size	2000
Discount factor	0.99
Learning rate	0.01
Network layer	2
Network size	64
Use reward to go	True
Baseline learning rate	0.01
Baseline gradient step	10
GAE lambda	0.95
Normalize advantages	True

My command line is as follows:

```
uv run src/scripts/run.py --env_name InvertedPendulum-v4 \
    -n 50 -b 2000 -eb 1000 \
    --discount 0.99 -lr 0.01 -l 2 -s 64 \
    --use_reward_to_go --use_baseline -blr 0.01 -bgs 10 \
    --gae_lambda 0.95 --normalize_advantages \
    --exp_name pendulum_tuned
```

Compared with the default setting, the tuned configuration reaches the maximum return 1000 at ~70k environment steps, demonstrating significantly improved sample efficiency. The main contributors are: the baseline with GAE which reduced variance while balanced bias, advantage normalization which ensures more stable updates, and a suitable batch size that enable more frequent updates per environment step while keeping variance small.

2. Learning curves comparison:


InvertedPendulum-v4: default vs tuned (avg return vs env steps)
pendulum
pendulum_tuned
Eval_AverageReturn
Train_EnvstepsSoFar
0
200
400
600
800
1000
0
100000
200000
300000
400000
500000